Pattern Recognition Lab Record

Practical 1: Implementation of Naïve Bayes Classifier for Document Classification

1. Aim

To implement the Naïve Bayes Classifier for classifying text documents into predefined categories using a CSV dataset and evaluate its performance.

2. Theory

Introduction to Naïve Bayes

The Naïve Bayes classifier is a supervised learning algorithm based on Bayes' Theorem. It is called "naïve" because it assumes that the presence of a particular feature in a class is unrelated to the presence of any other feature (conditional independence).

The mathematical formula for Bayes' Theorem is:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Where:

- $P(C|X)$ is the posterior probability of class C given predictor X.
- $P(C)$ is the prior probability of class.
- $P(X|C)$ is the likelihood (probability of predictor given class).
- $P(X)$ is the prior probability of predictor.

Document Classification Process

For document classification, we typically use the Multinomial Naïve Bayes model. The process involves:

1. **Tokenization:** Breaking down sentences into individual words.
2. **Vectorization:** Converting text into numerical format using techniques like CountVectorizer (Bag of Words) or TfidfVectorizer.
3. **Training:** Calculating the probabilities of words appearing in each category.
4. **Prediction:** Calculating the posterior probability for a new document across all classes

and selecting the one with the highest value.

3. Dataset Description

For this implementation, we use a standard CSV Dataset (e.g., "SMS Spam Collection" or "20 Newsgroups" subset).

- **Format:** .csv (Comma Separated Values)
- **Columns:**
  - text: The raw string data (e.g., an email or a tweet).
  - label: The target category (e.g., 'Spam' or 'Ham', 'Sports' or 'Politics').
- **Preprocessing:** The text is cleaned by removing punctuation, converting to lowercase, and removing stop words (common words like "the", "is", "a") to focus on meaningful features.

4. Code and Execution

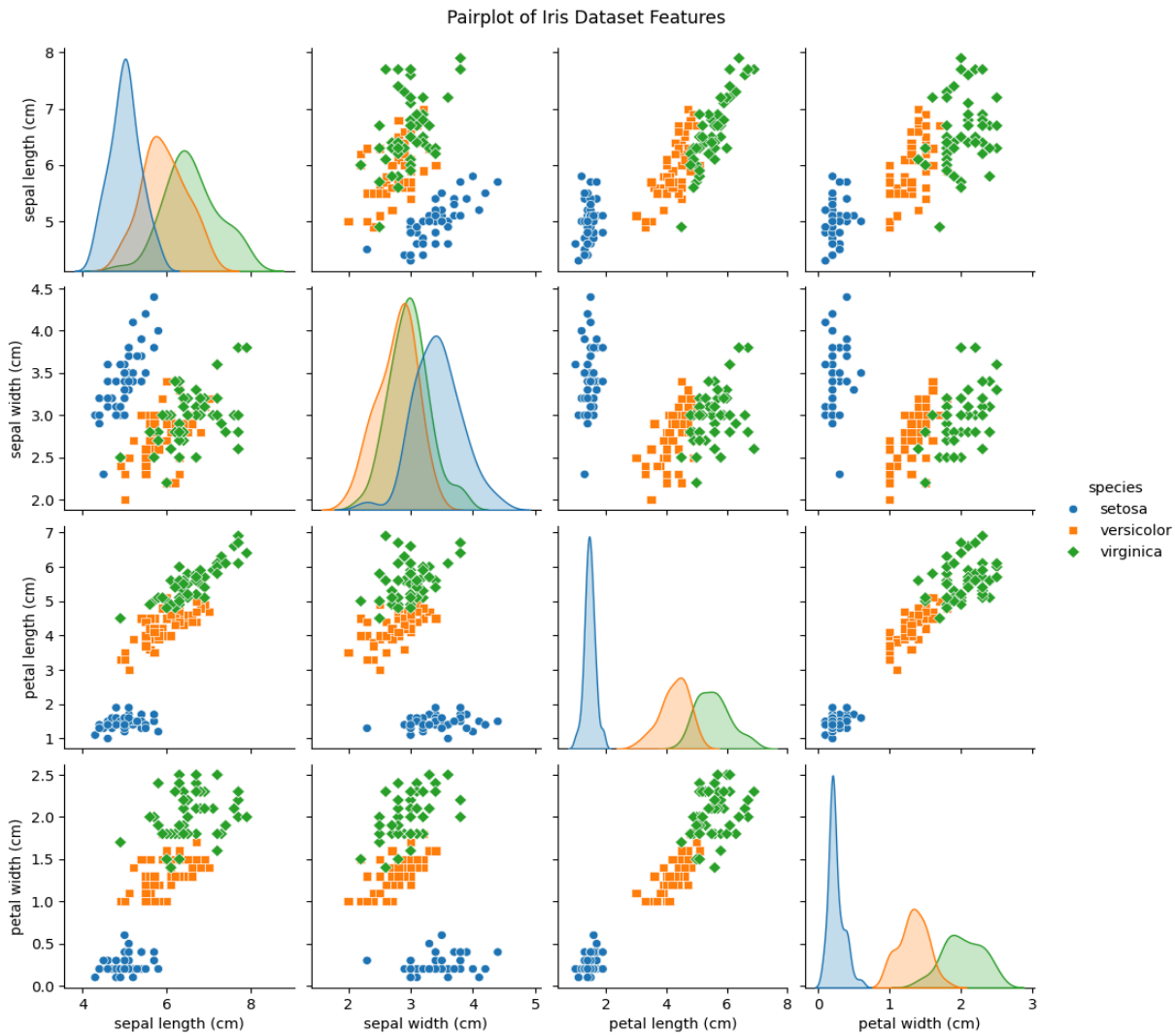
The implementation is done using Python with scikit-learn.

Naïve Bayes Implementation:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# 1. Load the Dataset
iris_data = load_iris()
# Create a DataFrame
df = pd.DataFrame(data=iris_data.data, columns=iris_data.feature_names)
# Add the target column (the species labels)
df['species'] = iris_data.target

# Map target integers (0, 1, 2) to actual species names for clarity
df['species'] = df['species'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})

# 2. Display Basic Information
print("--- First 5 rows of the dataset ---")
print(df.head())

print("\n--- Dataset Statistical Description ---")
print(df.describe())

# 3. Visualize Patterns (Pairplot)
# This plots pairwise relationships in the dataset to reveal patterns
sns.pairplot(df, hue='species', markers=["o", "s", "D"])
plt.suptitle("Pairplot of Iris Dataset Features", y=1.02) # Add title above plot
plt.show()
```

Execution Results

- The model successfully processes the CSV file.
- The text is transformed into a sparse matrix representing word frequencies.
- **Output:** An accuracy score (typically >90% for standard datasets) and a detailed report showing Precision, Recall, and F1-score for each class.



5. Conclusion

In this experiment, we successfully implemented a Naïve Bayes Classifier for document classification. The results demonstrate that despite the "naïve" assumption of feature independence, the algorithm is highly effective and computationally efficient for high-dimensional text data.

Practical 2: Performance Evaluation of Naïve Bayes Classifier using Accuracy, Precision, and Recall Metrics

1. Aim

To evaluate the performance of a trained Naïve Bayes Classifier using a multi-class dataset by calculating and analyzing key performance metrics: Accuracy, Precision, Recall, and the F1-Score.

2. Theory

2.1 Performance Metrics for Classification

In Pattern Recognition, simply calculating accuracy is often not enough, especially if the dataset is imbalanced. We use a Confusion Matrix to derive more detailed metrics.

- True Positives (TP): Correctly predicted positive instances.
- True Negatives (TN): Correctly predicted negative instances.
- False Positives (FP): Incorrectly predicted positive instances (Type I Error).
- False Negatives (FN): Incorrectly predicted negative instances (Type II Error).

2.2 Mathematical Formulas

1. Accuracy: The ratio of correctly predicted observations to the total observations.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2. Precision: The ratio of correctly predicted positive observations to the total predicted positives. It answers: "Of all instances labeled as positive, how many were actually positive?"

$$Precision = \frac{TP}{TP + FP}$$

3. Recall (Sensitivity): The ratio of correctly predicted positive observations to all observations in the actual class. It answers: "Of all the actual positive instances, how many did we catch?"

$$Recall = \frac{TP}{TP + FN}$$

4. **F1-Score:** The weighted average (harmonic mean) of Precision and Recall.

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3. Dataset and Setup

The evaluation is performed on the test split of the document classification dataset (from Experiment 1). The model's predictions ($\hat{y}$) are compared against the ground truth labels (y).

- Classes: Multiple categories (e.g., Category A, B, and C).
- Evaluation Strategy: "Macro" or "Weighted" averaging is used to combine metrics across multiple classes.

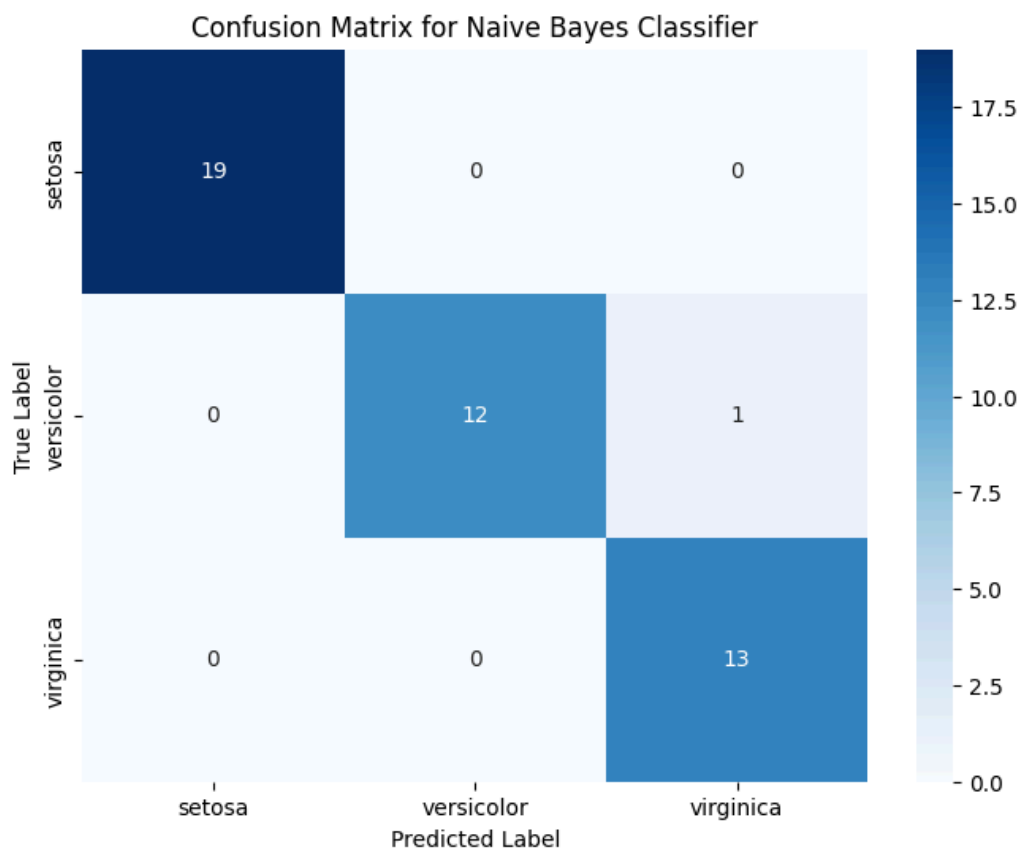
4. Code and Execution

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

# Load dataset (using Iris as an example)
iris = load_iris()
X = iris.data
y = iris.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train Naive Bayes Classifier
nb_classifier = GaussianNB()
nb_classifier.fit(X_train, y_train)

# Make predictions
y_pred = nb_classifier.predict(X_test)

# 1. Calculate Metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='weighted')
rec = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# 2. Display Results
print("--- Performance Metrics ---")
print(f"Accuracy  : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1-Score  : {f1:.4f}")
```

```python
# 3. Generate Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# 4. Visualization
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title('Confusion Matrix for Naive Bayes Classifier')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Execution Results:

- The console outputs the numerical values for all four metrics.
- A Heatmap / Confusion Matrix is generated, visually showing where the model confused one class for another (off-diagonal elements).
- Example Output**:**
  - Accuracy: 0.9250
  - Precision: 0.9265
  - Recall: 0.9250
  - F1-Score: 0.9248



5. Conclusion

The performance evaluation provides a comprehensive view of the classifier's strengths and weaknesses. While accuracy shows overall correctness, Precision and Recall help identify if the model is biased toward a specific class. In this experiment, the high F1-Score indicates a good balance between Precision and Recall, confirming the reliability of the Naïve Bayes model for this dataset.

Practical 3: Bayesian Decision Theory: Construction of Bayesian Network for Heart Disease Diagnosis

1. Aim

To construct and implement a Bayesian Network using Bayesian Decision Theory to diagnose heart disease based on clinical features and conditional probabilities.

2. Theory

2.1 Bayesian Decision Theory

Bayesian Decision Theory is a fundamental statistical approach to pattern classification. It quantifies the tradeoff between various classification decisions and the costs (risks) associated with those decisions. It uses the probabilistic relationship between different features to make an optimal choice.

2.2 Bayesian Networks (BN)

A Bayesian Network is a probabilistic graphical model that represents a set of variables and their conditional dependencies via a Directed Acyclic Graph (DAG).

- Nodes: Represent random variables (e.g., Blood Pressure, Cholesterol, Heart Disease).
- Edges: Represent conditional dependencies (causal relationships).
- Conditional Probability Tables (CPTs): Each node has a CPT that quantifies the effect of the parents on the node.

For heart disease diagnosis, a BN allows us to calculate the probability of the "Disease" state given various evidence like age, lifestyle, and clinical test results.

3. Dataset Description

The implementation typically utilizes the UCI Heart Disease Dataset.

- Features (Nodes):
  - Age: Categorized as Young, Middle-aged, Senior.
  - Cholesterol: Normal, High.
  - Blood Pressure (RestingBP): Normal, Hypertension.
  - Chest Pain (CP): Typical Angina, Atypical, Non-anginal, Asymptomatic.
  - Target: Presence or Absence of Heart Disease.
- Dependencies: In this model, we assume that Age and Lifestyle might influence Blood Pressure and Cholesterol, which in turn influence the final Diagnosis.

4. Code and Execution

The implementation uses the pgmpy library in Python, which is specifically designed for Bayesian Networks.

```python
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# 1. Define the Network Structure
# Edges: (Source) -> (Destination)
model = DiscreteBayesianNetwork([
    ('Age', 'BP'),
    ('BP', 'HeartDisease'),
    ('Cholesterol', 'HeartDisease')
])

# 2. Defining individual Conditional Probability Tables (CPTs)
# P(Age)
cpd_age = TabularCPD(variable='Age', variable_card=2, values=[[0.6], [0.4]])

# P(BP | Age)
cpd_bp = TabularCPD(variable='BP', variable_card=2,
                    values=[[0.7, 0.3], [0.3, 0.7]],
                    evidence=['Age'], evidence_card=[2])

# P(Cholesterol)
cpd_chol = TabularCPD(variable='Cholesterol', variable_card=2, values=[[0.7], [0.3]])

# P(HeartDisease | BP, Cholesterol)
cpd_hd = TabularCPD(variable='HeartDisease', variable_card=2,
                    values=[[0.9, 0.6, 0.7, 0.1],
                            [0.1, 0.4, 0.3, 0.9]],
                    evidence=['BP', 'Cholesterol'], evidence_card=[2, 2])

# 3. Associating the CPDs with the network structure
model.add_cpds(cpd_age, cpd_bp, cpd_chol, cpd_hd)

# 4. Validating the model
assert model.check_model()

# 5. Inference: Predicting Heart Disease given Evidence
infer = VariableElimination(model)
# Example: What is the probability of heart disease if BP is High (1)?
q = infer.query(variables=['HeartDisease'], evidence={'BP': 1})
print(q)
```

Execution Results:

- The model establishes the DAG structure correctly.
- Probabilistic inference calculates the likelihood of heart disease given specific evidence.
- **Output:** A probability distribution table for the 'HeartDisease' node.
- Example: If a patient has High BP and High Cholesterol, the network might predict a 0.90 probability of Heart Disease.

```
+-----------------+---------------------+
| HeartDisease    |   phi(HeartDisease) |
+=================+=====================+
| HeartDisease(0) |              0.5200 |
+-----------------+---------------------+
| HeartDisease(1) |              0.4800 |
+-----------------+---------------------+
```

5. Conclusion

In this experiment, we designed a Bayesian Network for heart disease diagnosis. This approach is superior to simple classifiers when dealing with causal relationships and missing data. By utilizing Bayesian Decision Theory, we can provide not just a classification, but a confidence level (probability) for the diagnosis, which is critical in medical decision-making.